

Semantic Cache

Similarity Thresholds

Conceptually, changing the similarity thresholds has tradeoffs:

- Accuracy -> Are we returning the correct cached answer?
- Cost Savings -> How many LLM calls do we avoid?

Lower threshold -> More Cache hits -> Cheaper, Riskier

Higher threshold -> Fewer Cache hits -> Safer, Expensive

Let's try with a medium size evaluation dataset

```
# Test dataset - Redesigned with more varied wording to produce diverse similarity scores
# Base queries use specific, concrete language
BASE_QUERIES = {
    "password_management": [
        "How do I change my password?",
        "I need to reset my login credentials.",
        "Where can I update my password?",
    ],
    "billing_management": [
        "How do I change my billing address?",
        "I want to update my payment method.",
        "Where is the billing settings page?",
    ],
    "account_settings": [
        "How do I update my profile information?",
        "I need to change my email address.",
        "Where can I edit my account details?",
    ],
    "product_features": [
        "What are the new features in the v2.0 update?",
        "Tell me about the latest product updates.",
        "What's new in version 2.0?",
    ],
}
```

```
# Test queries designed with progressively different wording to produce varied similarity scores:
# - Group A: High similarity (0.85-0.95) - semantically equivalent, different wording
# - Group B: Medium similarity (0.75-0.85) - related but different intent/scope
# - Group C: Low similarity (0.60-0.75) - completely different topics/domains
TEST_QUERIES = {
    "Group A - High Similarity (Should Hit at 0.75+)": [
        # Semantically equivalent to password_management base queries - different wording only
        ("I want to update my password", "password_management", "should_hit"),
        ("Reset my login credentials please", "password_management", "should_hit"),
        ("Where is the password change option?", "password_management", "should_hit"),
        # Edge cases - slightly different phrasing but same intent
        ("Can you help me change my password?", "password_management", "edge_case"),
        ("I'd like to modify my login password", "password_management", "edge_case"),
        ("How can I reset my account password?", "password_management", "edge_case"),
    ],
    "Group B - Medium Similarity (Should Hit at 0.75, Miss at 0.85+)": [
        # Related to account security but broader scope - should have medium similarity (0.75-0.85)
        ("I need help with account security settings", "password_management", "should_miss"), # Broader scope than just password
        ("What steps are involved in updating login credentials?", "password_management", "should_miss"), # More formal, process-oriented
        ("Can I modify my authentication information?", "password_management", "should_miss"), # Different terminology (auth vs password)
        ("Where would I find options to update my secret code?", "password_management", "should_miss"), # Different wording (secret code vs password)
    ],
    "Group C - Low Similarity (Should Miss at All Thresholds)": [
        # Completely different topics/domains - should have low similarity (0.50-0.75)
        # These should NOT match password_management base queries
        ("What are the latest product features and improvements in version 3.0?", "product_features", "should_miss"), # Product features - different domain
        ("Tell me about recent software updates and new releases for the application", "product_features", "should_miss"), # Software updates - different domain
        ("How do I update my credit card information for monthly billing?", "billing_management", "should_miss"), # Billing/payment - different domain
        ("I need to change my shipping address for package deliveries", "billing_management", "should_miss"), # Shipping - completely different
        ("What is the weather forecast for tomorrow?", "unrelated", "should_miss"), # Weather - completely unrelated
        ("How do I prepare a delicious pasta dish?", "unrelated", "should_miss"), # Cooking - completely unrelated
    ],
}
```

I will run the same queries at different thresholds:

1. 0.75 (Aggressive)
2. 0.80
3. 0.85 (Default)
4. 0.90 (Conservative)

For each threshold, I will run the queries in the same order and log:

- Cache hit or miss
- Similarity score
- Whether the cached response was “acceptable”

Below are some metrics that I collected after running this experiment

```

107 ("I need to change my shipping address for package deliveries", "billing_management", "should_miss"), #
108 ("What is the weather forecast for tomorrow?", "unrelated", "should_miss"), # Weather - completely unre
109 ("How do I prepare a delicious pasta dish?", "unrelated", "should_miss"), # Cooking - completely unrela

Problems Output Debug Console Terminal Ports GitLens

Estimated cost: $0.540000
Estimated savings: $0.300000
Similarity range: 0.905 - 0.947 (avg: 0.924)

=====
SUMMARY TABLE - Grouped by Model
=====

3_LARGE:
Threshold Hit Rate Acceptability LLM Calls Cost Savings Avg Sim
-----
0.75 12.5 % 100.0 % 25 $0.750000 $0.090000 0.788
0.80 6.2 % 100.0 % 26 $0.780000 $0.060000 0.811
0.85 0.0 % 100.0 % 28 $0.840000 $0.000000 N/A
0.90 0.0 % 100.0 % 28 $0.840000 $0.000000 N/A

3_SMALL:
Threshold Hit Rate Acceptability LLM Calls Cost Savings Avg Sim
-----
0.75 31.2 % 93.8 % 21 $0.630000 $0.210000 0.782
0.80 18.8 % 100.0 % 24 $0.720000 $0.120000 0.824
0.85 0.0 % 100.0 % 28 $0.840000 $0.000000 N/A
0.90 0.0 % 100.0 % 28 $0.840000 $0.000000 N/A

ADA_002:
Threshold Hit Rate Acceptability LLM Calls Cost Savings Avg Sim
-----
0.75 100.0 % 37.5 % 2 $0.060000 $0.780000 0.862
0.80 87.5 % 50.0 % 6 $0.180000 $0.660000 0.878
0.85 62.5 % 68.8 % 12 $0.360000 $0.480000 0.906
0.90 50.0 % 81.2 % 18 $0.540000 $0.300000 0.924

=====
Similarity Score Ranges (for cache hits):

text-embedding-3-large:
Threshold 0.75: Min=0.766, Max=0.811, Avg=0.788
Threshold 0.80: Min=0.811, Max=0.811, Avg=0.811

text-embedding-3-small:
Threshold 0.75: Min=0.753, Max=0.821, Avg=0.782
Threshold 0.80: Min=0.810, Max=0.842, Avg=0.824

text-embedding-ada-002:
Threshold 0.75: Min=0.757, Max=0.947, Avg=0.862
Threshold 0.80: Min=0.834, Max=0.947, Avg=0.878
Threshold 0.85: Min=0.864, Max=0.947, Avg=0.906
Threshold 0.90: Min=0.905, Max=0.947, Avg=0.924

```

What can we derive?

Text-embedding-3-small: **balance reuse, predictable thresholds**(reuse stops clearly as the threshold increases), **safe costs** (reduce LLM calls while favoring correctness over recall)
So it cuts LLM calls without reusing risky matches

Text-embedding-ada-002: **aggressive reuse, sensitive tuning**(Small threshold changes cause large behavior changes), **high risk** (maximize savings but can reuse loosely related to answers)

Text-embedding-3-large: **conservative reuse, high precision, low savings impact**(Reuse stops clearly as the threshold increases).

Different embedding models produce similarity scores with different distributions and dynamic ranges. As a result, optimal similarity thresholds are model-specific. For example, text-embedding-ada-002 produces higher and more widely distributed similarity scores, allowing finer-grained threshold tuning, whereas text-embedding-3-small and 3-large produce tighter similarity clusters, causing reuse to collapse at higher thresholds.

There is no universally correct similarity threshold or embedding model. However, based on empirical evaluation:

- text-embedding-3-small provides the most predictable, conservative, and cost-effective behavior for semantic caching.
- A threshold around 0.80 offers a reasonable default on the evaluated dataset, balancing safe reuse and cost savings.
- Both the threshold and model choice should be revisited as query diversity or application requirements change.

Can we do more improvement after this? For text-embedding-3-small, it has a 100% acceptability rate but the hit rate is still low. How can we **increase** it? After some research, we can optimize the query text by preprocessing it.

The text we embed has a massive impact on the vector similarity. We must ensure to focus the embedding model on the **core meaning** rather than **filler** words. Remove **"Semantic Noise"**: Filter out common, non-meaning-carrying phrases like greetings ("How do I", "Can you show me", "I want to", "please", "thank you"). This helps queries like "How do I change my password?" and "Reset my password" produce highly similar vectors. Below is the result:

3_LARGE:						
Threshold	Hit Rate	Acceptability	LLM Calls	Cost	Savings	Avg Sim
0.75	43.8	% 87.5	% 19	\$0.570000	\$0.270000	0.823
0.80	18.8	% 100.0	% 24	\$0.720000	\$0.120000	0.890
0.85	18.8	% 100.0	% 24	\$0.720000	\$0.120000	0.956
0.90	12.5	% 100.0	% 26	\$0.780000	\$0.060000	1.000
3_SMALL:						
Threshold	Hit Rate	Acceptability	LLM Calls	Cost	Savings	Avg Sim
0.75	50.0	% 87.5	% 18	\$0.540000	\$0.300000	0.838
0.80	25.0	% 100.0	% 22	\$0.660000	\$0.180000	0.905
0.85	18.8	% 100.0	% 23	\$0.690000	\$0.150000	0.933
0.90	18.8	% 100.0	% 24	\$0.720000	\$0.120000	0.933
ADA_002:						
Threshold	Hit Rate	Acceptability	LLM Calls	Cost	Savings	Avg Sim
0.75	87.5	% 50.0	% 5	\$0.150000	\$0.690000	0.872
0.80	75.0	% 62.5	% 7	\$0.210000	\$0.630000	0.887
0.85	68.8	% 68.8	% 10	\$0.300000	\$0.540000	0.910
0.90	50.0	% 87.5	% 18	\$0.540000	\$0.300000	0.942

We can see a clear improvement, especially in the 3-small. From this experiment itself, I decided to use **3-small with similarity threshold 0.75**, since it has the maximum saving, high acceptability and good hit rate. ADA-2 has a high hit rate but it is very risky with 50% acceptability.

While ADA-002 at 0.9 matches 3-small on metrics, after some research, it is optimized for retrieval rather than answer equivalence, making reuse decisions less stable. 3-small provides more predictable semantic discrimination and safer reuse at the same cost, which is why it is preferred for production.

Redis

What do we store in our redis?

- Cached query responses (with embeddings)
- Cached embeddings (to avoid regenerating them)
- Cache key index (set of all cache entry keys)
- Statistics counters (request counts, cache hits/misses, etc.)

The Process: From Query to Response

Step 1: Query normalization and embedding generation

When a query arrives:

1. Normalize the query by converting to lower case and trim any whitespace
2. Check **embedding cache** to look for existing embedding in Redis
 - a. Key pattern: `embed:{model_name}:{sha256_hash_of_normalized_query}`
 - b. If found, use the cache embedding and use the answer.
 - c. If not found, generate new embedding with OpenAI embedding model.
3. Store embedding for future use, we also use TTL (configurable, but typically longer than the response cache.)

Why would we want to cache embedding? Save Cost from generating new embeddings from identical/similar query.

Step 2: Semantic Similarity Search

After we have the embedding, search for similar cached responses,

1. Retrieve all cache entry keys (set of all cached responses)
2. For each key, check their entries (JSON with query, embedding, response, metadata).
3. For each entry, calculate [cosine similarity](#) while tracking the best match (highest similarity score)
4. While doing this search, we also clean up expired entries (TTL passed) and remove it from the cache entry keys set to keep it clean.

5. After calculating and getting the highest similarity score from all entries, we grab the **similarity threshold**, and if **highest similarity score > threshold**, that is a cache hit and we update the statistics of cache hit and skip the LLM call (cut cost)
6. If **highest similarity score < threshold**, that is a cache miss and we go to step 3, LLM call.

Step 3:

If no similar cached response found or (forceRefresh is true):

1. Call LLM and generate responses via our used OpenAI Model.
2. Store response and generate new entry key with value json containing query, embedding, response, metadata, TTL
3. Add the key to the Cache key index to enable future similarity searches.
4. Update statistics (cache miss ++ and LLM call count.)

Persistence Implementation

The "**dual persistence strategy**" in Redis, formally known as hybrid persistence, involves enabling both the RDB (Redis Database) snapshotting and the AOF (Append-Only File) logging.

```
# RDB persistence (snapshots)
# Save the dataset to disk if at least one key changed and the given number of seconds passed
save 900 1      # Save after 900 sec if at least 1 key changed
save 300 10     # Save after 300 sec if at least 10 keys changed
save 60 10000   # Save after 60 sec if at least 10000 keys changed

# AOF persistence (Append Only File) - more durable
```

```
# AOF persistence (Append Only File) - more durable
appendonly yes
appendfsync everysec # fsync every second (good balance between performance and durability)
```

How snapshots and AOF (operation logs) work together? They rebuild state to any desired moment, balancing fast recovery with minimal data loss.

We also mount the docker volume for redis so it persists independently from the container lifecycle.

```
redis:
  volumes:
    - redis_data:/data # Persistent volume
```

What does this code snippet do? We are telling Docker:

- "Run this Redis Container."
- "Create or find a Volume named redis_data on my host machine."
- "Mount that volume into the /data directory inside the Container."

Now, when Redis performs its persistence strategy (writing dump.rdb or appendonly.aof), it writes those critical files to the /data directory inside the container, which seamlessly redirects the data path to the safe, persistent redis_data volume on the host machine. So if something happens to the container, we use this volume to a new container with the same volume path to use the previously saved data files (plug and play).

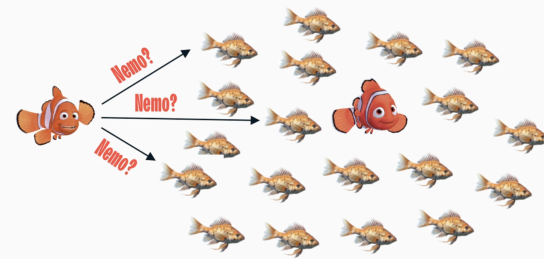
Scale the system

Identified bottlenecks when I scanned the current state of the system:

1. **O(n) Linear Scan** - the system iterates through all cache keys, making it O(n) where n = number of cached entries. This becomes the primary bottleneck as cache grows.
2. **Synchronous Operations** - FastAPI endpoints are synchronous, not leveraging async capabilities for I/O-bound operations (Redis, OpenAI API calls).
3. **No Connection Pooling** - Each request creates new Redis clients, causing connection overhead.
4. **Sequential API Calls** - Embedding and LLM calls are sequential with no batching or parallelization.
5. **No Horizontal Scaling** - Single API instance with no load balancing.
6. **No Request-Level Caching** - We currently generate embedding for all query instead of checking the query to entries directly, since we hash, look up is O(1).

Linear search: O(N)

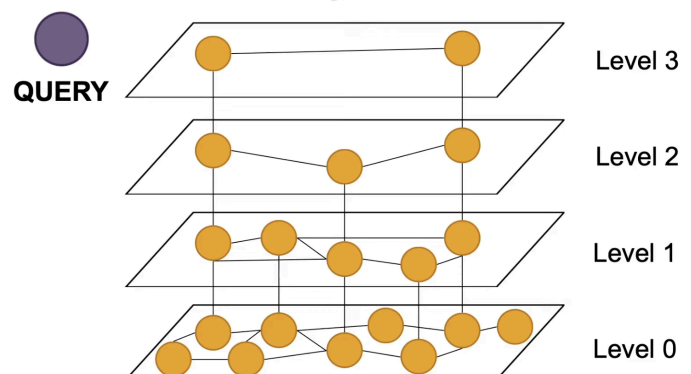
Performance: slow



Scaling strategy that can be implemented:

1. Replace linear scan with vector search by using external vector DB, since I used [Weviate](#) (Uses [HNSW](#) Algorithm O(log n)) in my current work, let's use that. The implementation will be:
 - a. Add vector DB client
 - b. Store embeddings in vector DB, keep responses in Redis
 - c. Query vector DB for similar embeddings, then fetch responses from Redis
2. Implement Async Operations from synchronous endpoints to handle concurrent requests efficiently.
3. Add singleton connection pooling for redis to reduce connection number.
4. Add fast response cache (in-memory or Redis) before semantic search to handle **exact** duplicate queries. Since we hash(query), redis will do O(1) lookup to exact previous hash(query) and return the response.
5. Batch embedding generation when possible (OpenAI batch API)
6. Add rate limiting middleware and circuit breakers for API calls to protect against overload and improve stability under traffic spikes.

Hierarchical Navigable Small Worlds



For infrastructure scaling, we can add load balancers and automatic scaling multiple api instances, and Terraform can be used to define, provision, and manage the load balancers and auto-scaling instances configurations as part of an Infrastructure as Code (IaC) approach. It does not perform the actual load balancing or instance replication itself, but it automates the setup and configuration of the cloud provider's services that handle these tasks.

Parallel Processing and Batching Implementation

After identifying some processing bottlenecks (sequential calls, no batching, cache inefficiency), I decided to implement **parallel processing** and **batching** for **OpenAI API** calls to improve throughput and reduce latency. The call processes up to 2048 (Configurable) texts in a single request, reducing round trips.

Also added with semaphore-based rate limiting to prevent API limit violations (set default to 10, configurable).

```
# Without rate limiting (BAD - could hit rate limits)
queries = ["query1", "query2", "query3", ..., "query100"]
# All 100 queries fire at once → OpenAI rate limit error! ❌

# With rate limiting (GOOD - controlled concurrency)
queries = ["query1", "query2", "query3", ..., "query100"]
# Only 10 queries run concurrently (MAX_PARALLEL_LLM_CALLS=10)
# Others wait in queue → No rate limit errors! ✅
```

Enhanced SemanticCache with **get_or_create_embeddings_batch()** to check cache in parallel, batch-generate missing embeddings, and cache results concurrently.

```
# Build cache keys for all queries
cache_keys = [
    f"embed:{model_name}:{hash(norm)}"
    for norm in normalized_queries
]

# Check cache for ALL queries in parallel ⚡
cached_results = await asyncio.gather(*[
    self._redis.get(key) for key in cache_keys
])

# Now we know which ones are cached and which need generation
for i, cached in enumerate(cached_results):
    if cached:
        result_embeddings[i] = json.loads(cached) # Use cached
    else:
        embeddings_to_generate.append(queries[i]) # Need to generate
```

These changes improve **scalability**, reduce **latency**, and better utilize **async I/O** from a single endpoint.

```
# Batch endpoint for processing multiple user queries
@router.post("/query/batch")
async def batch_query_endpoint(payload: BatchQueryRequest):
    queries = payload.queries # ["q1", "q2", ..., "q50"]

    # Get embeddings for all queries (parallel cache + batch generation)
    embeddings = await cache.get_or_create_embeddings_batch(queries)

    # Get LLM completions for all queries (concurrent with rate limiting)
    completions = await openai_client.get_completions_batch(queries)

    return {"results": completions}
```

Cosine Similarity

Measure angle between two embedding vectors to get their distance in the vector space to get their similarity context. The beauty of the cosine similarity formula is that it generalizes perfectly to any N-dimensional space. In our case, **text-embedding-3-small** returns 1536 dimensions.

$$\text{Similarity} = \sum_{i=1}^D A_i B_i$$

where:

- A and B are your two vectors.
- D is the number of dimensions (e.g., 1536).
- A_i and B_i are the values at each dimension i .

Range from -1.0 to 1.0

A cosine similarity score close to 1 indicates high similarity (the vectors point in the same direction).

A score close to 0 indicates low or no similarity (orthogonal vectors).

A score close to -1 indicates high dissimilarity (vectors point in opposite directions).